Phase 6 developer handoff

Delivery boundary

This slice implements tenant-owned source tasks, explicit source scope, private Git snapshots, contracts, immutable patch proposals, human staging review, private rollback and a fail-closed local Docker build adapter. Docker Desktop 4.94.0 is configured in Linux/amd64 mode, and a pinned Java 21/Maven image successfully built and tested a controlled fixture with networking disabled. Customer-specific dependency caches, analyzed-artifact reproduction and the broader Phase 6 gates remain unvalidated. Phase 6's exit gate is open.

The master specification remains authoritative. This slice does **not** complete automatic characterization-test generation, full generated-code SAST/SCA, architecture reanalysis, transformation graph projection, final approval/merge, post-merge revert, distributed task scheduling or the full planned task state machine. These remain explicit work items, not implied completed features. Phase 7 implementation has not started.

Code and storage boundaries

Component	Responsibility
`backend/app/transformation/schemas.py`	Strict request schemas, bounded edits, portable paths and filtered metadata
`models.py` / migration `0007`	Task, changeset, behavioral-contract and build-attempt records
`service.py`	Current evidence/plan checks, state transitions, authorization bindings, review and recovery
`workspace.py`	Private Git setup, SHA manifests, containment/link checks, bounded staging and rollback
`sandbox.py`	Local Docker CLI only, fixed build recipe, resource flags, bounded tar/XML handling
`routes.py`	Tenant-scoped API, source read checks and safe error responses
`frontend/src/transformation/TaskWorkspace.tsx`	Source-task preparation, contracts, patch review, build history and controls
`backend/app/access/gateway.py`	Existing audited source gateway with an internal bounded-task scope

Data path: `DATA_DIR/transformations/<tenant>/<task-id>/baseline` plus `worktree`. The latter has its own `.git`; it is not a Git linked worktree attached to the customer's repository. Local Git is used for a pinned private baseline and branch, while reviewed changes are tracked by explicit manifests and immutable changeset documents. Staging means writing approved bytes in that private directory, not merging or committing to customer Git. Source snapshots and changeset content remain in local storage until operator-managed retention removes them. This slice does not add encryption at rest or an automatic source-task purge; protect the data directory with the deployment's filesystem access controls and storage policy.

Migration `0006 -> 0007` is additive: `transformation_tasks`, `transformation_changesets`, `transformation_contracts`, `transformation_builds`. Existing static/runtime/evidence/planning records are not rewritten. Task fingerprints are unique per tenant and include actor, version and normalized request options, including optional request key. Identity-scoped sessions enforce tenancy; related version access also enforces runtime permissions when necessary.

Request and authorization flow

1. Existing authentication, CSRF, permission and durable request-intent audit middleware runs first. Reads require analysis-read; writes require analysis-write. Source-sensitive task endpoints recheck current `SOURCE_READ`, task policy and explicit repository grants.
2. The selected current architecture version, application, registered repository commit and planning proposal are bound into the task. The latest planning review must be `APPROVED_PLANNING`.
3. The source gateway reads literal regular blobs at the registered commit. It enforces path grants/protected paths and task-specific file/byte limits. It records the permitted manifest durably before returning source bytes.
4. Source text and patch text pass the existing offline secret/PII/prompt-injection filters. No model call occurs. Raw source does not enter audit records, build logs or general analysis exports.
5. Task mutations use the existing process lock and optimistic revisions. The lock is released during container execution and authorization is checked again afterward.

`read_context(..., task_scope=True)` is an internal service option, not an accepted public context-request field. Existing read-only context budgets and behavior remain unchanged. The gateway rolls back its read transaction before committing its independent audit event. Assign durable mutation results **after** calls through that boundary; otherwise pending ORM changes can be lost.

Task-list metadata is visible to authorized version readers; full task documents, source and patch details additionally require current source scope. Source reads revalidate the original registered commit and captured file hashes, so removing the original repository or revoking its grant blocks later task access. Administrative recovery after source-access revocation is not yet a separate workflow.

States and approval semantics

State	Meaning / allowed progression
`PREPARING`	Persisted preparation intent before filesystem work; succeeds to PLANNED or fails
`PLANNED`	Private baseline prepared; contracts, patch proposals and baseline builds available
`READY`	Baseline artifact/tests and reviewed critical contracts satisfy preparation gaps
`STAGING`	Durable staging intent before approved byte writes
`STAGED`	One reviewed initial patch written privately
`BUILDING`	A baseline or candidate build attempt is active
`VALIDATION_BLOCKED`	Staged candidate still lacks final architecture/security acceptance
`PAUSED`	Progression paused; an existing build must finish before resume/rollback
`FAILED`	Interrupted or failed preparation; inspect before recheck/retry/rollback
`ROLLED_BACK`	Private baseline restored; terminal task

Every event records actor, time, action, revision, note and relevant patch/build identity. Task histories are bounded to 200 events; rollback remains possible at the limit. Changesets and build attempts are limited to 50 each; contracts to 100. This is a single-process prototype: do not run multiple API workers against these tasks. A distributed lease/claim protocol remains future work.

Patch documents include expected per-file hashes, full replacement contents, evidence IDs, baseline digest, risk and validation requirements. A task supports one initial staged changeset. Risk is a conservative filename/size/deletion heuristic, not semantic vulnerability analysis. Human staging approval binds exact patch hash, reviewer, source grant, current policy revision and 24-hour expiry. HIGH risk requires a different human security administrator. Final transformation approval is unavailable.

Startup marks PREPARING/STAGING/BUILDING tasks and RUNNING builds failed without automatically executing anything. Before staging, persist the approved before/after hash alternatives. Recovery rollback accepts only those recorded alternatives, rejects unrelated tampering, and never recursively deletes a computed path. Partial writes with unknown hashes require operator inspection.

Build adapter and operator setup

`infra/docker/Dockerfile` builds the application. **It does not provide the customer build sandbox.** The separate template is `infra/docker/build-sandbox.Dockerfile`. The packaged application image is not yet certified to operate this adapter; the current adapter targets the local host-launched API and an explicitly local Docker daemon.

Prerequisites:

- Local Docker CLI and reachable Linux-container daemon at the configured fixed local pipe/socket. Remote Docker contexts and inherited `DOCKER_*` values are not used.
- Operator-approved immutable Maven and JDK base-image digests already provisioned locally.
- A reviewed `maven-repository/` build context containing all dependency and plugin artifacts required by the target. Exclude credentials, settings files, private source and unnecessary cached artifacts. Choose a JDK compatible with the target.
- For the Phase 7 image, also copy the server-owned `infra/docker/refactor` directory into the context as `refactor/`. Helper provisioning downloads its pinned JavaParser/build dependencies from Maven Central; do not include customer inputs. Customer builds remain offline at runtime.
- An image with `/bin/sh`, `sleep`, `cp`, Maven and JDK, readable offline repository at `/opt/m2`, and no image-declared anonymous writable volumes. Test non-root access and Windows bind-mount sharing before acceptance.

Example operator build from a prepared context (replace both placeholder digests with verified values):

```
docker build --pull=false --build-arg MAVEN_BASE=maven@sha256:<verified-digest> --build-arg JDK_BASE=eclipse-temurin@sha256:<verified-digest> -
f C:\path\to\vfunction\infra\docker\build-sandbox.Dockerfile -t archintel-build:local C:\approved-build-context
docker image inspect archintel-build:local --format '{{.Id}}'
```

Set `BUILD_SANDBOX_IMAGE` in the local operator configuration to the returned full `sha256:<64 hex>` image ID. `BUILD_TIMEOUT_SECONDS` defaults to 180 and accepts 10..600. Restart the API after changing configuration. Enable workspace source-task/build policies and explicit grants only for the intended users/repository. The current local configuration pins `sha256:9847fa2c085e6b1dfc3a7921d16184ee1925c20fc50b50d76fd8433ba8acfd81`; it includes the reviewed offline Maven cache captured after the Concord 2.46.1-SNAPSHOT build; rebuild it whenever a target application requires dependencies absent from that cache. The adapter uses `/opt/m2/repository` read-only and does not copy the cache into `/work`; `/work` remains a bounded writable tmpfs for source and build outputs.

Read-only prerequisite check from the workspace root:

```
.\.runtime\python\tools\python.exe scripts/check_build_sandbox.py
```

Exit 0 means the local image/daemon prerequisite check passed, **not** that isolation or the customer build passed. Exit 1 means unavailable. Actual execution acceptance is still required.

The server creates a generated container name with `--pull=never`, no network, non-root UID/GID 1000, read-only root, all capabilities dropped, no-new-privileges, 256 PIDs, 2 CPUs, 1 GiB memory/no additional swap, no healthcheck and no log driver. Only the task snapshot is bound read-only. Writable tmpfs: `/tmp` 64 MiB and `/work` 512 MiB. No customer home, host socket, credentials or checkout is mounted.

The fixed command copies input and offline Maven cache into `/work`, then runs:

```
mvn -o -B -Dmaven.repo.local=/work/.m2 -Dmaven.test.failure.ignore=false -DskipTests=false -Dmaven.test.skip=false package
```

Customer Maven plugins execute only within this container boundary. Repository wrappers and user-supplied shell commands are never invoked on the host. Docker itself remains a trusted privileged operator dependency; these configuration flags are not proof of resistance to kernel/container escapes. See the primary Docker documentation for runtime constraints and tmpfs mounts.

The adapter bounds captured command output and parses report tar streams from `docker exec tar` in memory without extracting paths. Surefire XML rejects DTD/entities, nonregular reports and oversize input. Artifact bytes are streamed through `docker exec cat` with a 32 MiB limit and a regular-file/non-symlink check inside the container. Baseline bytes and raw output are discarded after deriving structured results and SHA-256. Phase 7 candidate builds retain validated archives under task-owned storage for re-analysis; see the Phase 7 developer guide for binding, integrity and retention rules. The container is removed in `finally`; if the API dies, its fixed 660-second lifetime bounds execution, but stopped-container reconciliation/cleanup still needs an operator. No startup cleanup guarantee is claimed.

Evidence and validation semantics

- Complete snapshot is exact equality between selected file paths and committed file inventory; partial scope cannot build.
- Baseline success requires command success, at least one non-skipped case, no reported failures/errors and exact match to the captured analyzed artifact SHA-256.
- Reviewed critical contracts require passing case-name evidence from the latest passing baseline. Test-file hashes are recorded. This does not verify test assertion quality or prove full business equivalence.
- Candidate PASSED records only the bounded build/test result. Task state remains VALIDATION_BLOCKED. Candidate contract comparison, architecture change measurement and comprehensive generated-code security gates remain open.
- Secret detection uses the existing established `detect-secrets` detectors plus local filters. This is not a SAST/SCA or dependency-vulnerability scan. Architecture/security/behavior validation fields on changesets remain `NOT_RUN` until those dedicated validators exist.
- Customer-controlled Maven plugins can affect reports; do not treat their XML as independent behavioral certification. Trusted acceptance fixtures and stronger test provenance remain required.

API surface

All paths below use `/api/v1` and existing session/CSRF conventions.

Method/path	Action
GET `/transformation/capabilities`	Read local sandbox availability
POST/GET `/architecture-versions/{version}/tasks`	Prepare/list version tasks
GET `/transformation/tasks/{task}`	Task, history, contracts, changesets, builds and gaps
GET `/transformation/tasks/{task}/source/{path}`	Authorized current private-file bytes as UTF-8 JSON
POST `/transformation/tasks/{task}/contracts`	Add captured-test behavioral assertion
POST `/transformation/tasks/{task}/contracts/{contract}/review`	Immutable human review
POST `/transformation/tasks/{task}/changesets`	Propose bounded exact edits
GET `/transformation/tasks/{task}/changesets/{change}`	Exact diff and validation status
POST `/transformation/tasks/{task}/changesets/{change}/review`	Approve private staging or reject
POST `/transformation/tasks/{task}/changesets/{change}/stage`	Apply approved bytes privately
POST `/transformation/tasks/{task}/actions`	Pause, resume, recheck or rollback

Method/path	Action
POST `/transformation/tasks/{task}/builds/{BASELINE|CANDIDATE}`	Synchronous bounded build attempt

Mutations with revision schemas require `expected_revision` and a filtered note. Contract addition is append-only and does not require a client revision. API responses can record failed preparation/build attempts successfully: inspect returned state/error fields rather than assuming HTTP 2xx means the operation passed. There is no merge, deployment or autonomous-model-execution endpoint.

Remaining acceptance and development work

1. Provision the local sandbox and a supported complete source fixture with reproducible artifact and existing tests. Run real baseline/candidate success and failure cases, and verify no host/source writes.
2. Exercise no-egress, non-root, readonly mount/root, PID/memory/timeout limits, hostile report handling and cleanup on the actual engine; record image/JDK/Maven versions and measurements.
3. Complete the broader Phase 6 contract/characterization, task lineage and established SAST/SCA integrations from the master specification. Tool unavailable must stay NOT_RUN/BLOCKED, never PASS.
4. Only after Phase 6's acceptance gate and user verification, continue the controlled Phase 7 refactor, reanalysis/comparison and final approval work.

Run `backend/tests/test_transformation.py` for source isolation, authorization, review invalidation, recovery and structured-result tests. The passing build/artifact-gate unit tests mock `sandbox.run`; they do not replace real container acceptance. PostgreSQL integration uses a temporary schema and controlled source fixture, preserving a staged patch across API restart before rollback. Browser validation exercises real Git/private-file workflows against an isolated application database. See verification for recorded results and limitations.